1.Aim of the practical: Implement CRUD operation with database on NodeJS with MongoDB/MySQL .

2.Objective: Implement CRUD operations in a Node.js backend using MongoDB/MySQL for data storage and management.

3.Algorithm/Flowchart:

React.js:

```
import React, { useState, useEffect } from "react";
import axios from "axios";

function App() {
  const [users, setUsers] = useState([]);
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [age, setAge] = useState("");
  const [editId, setEditId] = useState(null);

  useEffect(() => {
    fetchUsers();
  }, []);

  const fetchUsers = async () => {
    try {
      const res = await axios.get("http://localhost:5000/users");
      setUsers(res.data);
    } catch (error) {
      console.error("Error fetching users:", error);
    }
  };

  const handleSubmit = async (e) => {
```

```
    e.preventDefault();
    try {
      if (editId) {
        await axios.put(`http://localhost:5000/users/${editId}`, { name, email, age });
      } else {
        await axios.post("http://localhost:5000/users", { name, email, age });
      }
      setName("");
      setEmail("");
      setAge("");
      setEditId(null);
      fetchUsers();
    } catch (error) {
      console.error("Error adding/updating user:", error);
    }
  };

  const handleDelete = async (id) => {
    if (window.confirm("Are you sure you want to delete this user?")) {
      await axios.delete(`http://localhost:5000/users/${id}`);
      fetchUsers();
    }
  };

  const handleEdit = (user) => {
    setName(user.name);
    setEmail(user.email);
    setAge(user.age);
    setEditId(user.id);
  };

  return (
    <div style={{ textAlign: "center", padding: "20px", fontFamily: "Arial, sans-serif" }}>
      <h2> 🔥 CRUD App (React + Node + MySQL)</h2>
      <form onSubmit={handleSubmit} style={{ marginBottom: "20px" }}>
        <input type="text" placeholder="Name" value={name} onChange={(e) => setName(e.target.value)}
required />
        <input type="email" placeholder="Email" value={email} onChange={(e) => setEmail(e.target.value)}
required />
        <input type="number" placeholder="Age" value={age} onChange={(e) => setAge(e.target.value)}
required />
        <button type="submit" style={{ background: editId ? "#f39c12" : "#2ecc71", color: "#fff" }}>
          {editId ? "Update" : "Add"} User
        </button>
      </form>
      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>ID</th>
```

```
            <th>Name</th>
            <th>Email</th>
            <th>Age</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody>
          {users.length > 0 ? (
            users.map((user) => (
              <tr key={user.id}>
                <td>{user.id}</td>
                <td>{user.name}</td>
                <td>{user.email}</td>
                <td>{user.age}</td>
                <td>
                  <button onClick={() => handleEdit(user)}>✏ Edit</button>
                  <button onClick={() => handleDelete(user.id)}>🗑 Delete</button>
                </td>
              </tr>
            ))
          ) : (
            <tr>
              <td colSpan="5">No Users Found!</td>
            </tr>
          )}
        </tbody>
      </table>
    </div>
  );
}

export default App;
```

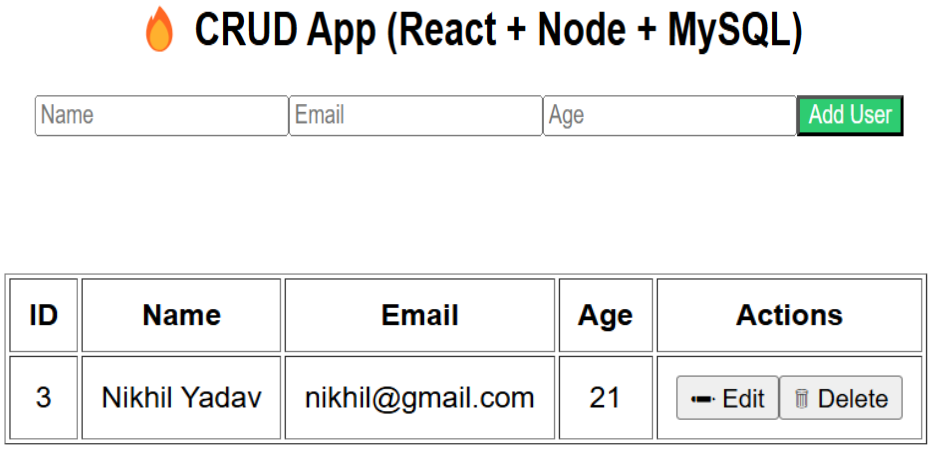
Node.js:

```
const express = require("express");
const bodyParser = require("body-parser");
const cors = require("cors");
const mysql = require("mysql2");
require("dotenv").config();

const app = express();
const PORT = process.env.PORT || 5000;
// app.listen(PORT, () => console.log(`Server running on port ${PORT}`));

// Middleware
// app.use(cors());
app.use(cors({ origin: "http://localhost:3000" })); // React runs on port 3000
```

```
app.use(bodyParser.json());

// MySQL Database Connection
const db = mysql.createConnection({
    host: process.env.DB_HOST,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME,
});

// Connect to MySQL
db.connect((err) => {
    if (err) {
        console.error("❌ Database connection failed:", err.message);
        process.exit(1); // Stop server if DB connection fails
    }
    console.log("✅ Connected to MySQL database.");
});
// CREATE User (INSERT)
app.post("/users", (req, res) => {
    const { name, email, age } = req.body;
    if (!name || !email || !age) {
        return res.status(400).json({ error: "All fields are required." });
    }
    const sql = "INSERT INTO users (name, email, age) VALUES (?, ?, ?)";
    db.query(sql, [name, email, age], (err, result) => {
        if (err) return res.status(500).json({ error: err.message });
        res.status(201).json({ message: "User created successfully", userId: result.insertId });
    });
});

// READ All Users
app.get("/users", (req, res) => {
    db.query("SELECT * FROM users", (err, results) => {
        if (err) return res.status(500).json({ error: err.message });
        res.json(results);
    });
});

// READ Single User
app.get("/users/:id", (req, res) => {
    const { id } = req.params;
    db.query("SELECT * FROM users WHERE id = ?", [id], (err, result) => {
        if (err) return res.status(500).json({ error: err.message });
        if (result.length === 0) return res.status(404).json({ message: "User not found" });
        res.json(result[0]);
    });

// DELETE User
```

```
app.delete("/users/:id", (req, res) => {
  const { id } = req.params;
  db.query("DELETE FROM users WHERE id = ?", [id], (err, result) => {
    if (err) return res.status(500).json({ error: err.message });
    if (result.affectedRows === 0) return res.status(404).json({ message: "User not found" });
    res.json({ message: "User deleted successfully" });
  });
});

// Start Server
app.listen(PORT, () => {
  console.log(`🚀 Server running on http://localhost:${PORT}`);
});
```

4.Result/Output:



Learning outcomes (What I have learnt):

- Understand how to integrate Node.js with MongoDB for database operations.
- Implement CRUD functionalities (Create, Read, Update, Delete) using Express.js.
- Learn to handle API requests and responses efficiently in a backend application.